AI-Assisted Parallelization of bzip2 Compression

Final Project — Parallel Programming, Spring 2026

Lin Ching-Yun, Hsieh Tien-Hua, Cheng Yu-Shiang
Department of Computer Science and Information Engineering
National Taiwan University
{b12902116, b11902088, b12902025}@ntu.edu.tw

***Abstract*—Data compression is a critical component of high-performance computing, with algorithms like bzip2 prized for their high compression ratios . bzip2 internally processes data in blocks, making it a potential candidate for parallelization. However, effectively parallelizing this process requires careful management of output ordering, synchronization overhead, and I/O bottlenecks. This paper explores the efficacy of Large Language Models (LLMs) in automating the parallelization of a sequential bzip2 compressor (pbzx). We evaluate an AI-assisted development workflow across three progressive stages of guidance: naive prompting, constraint-guided design, and profiling-guided optimization. By comparing these AI-generated implementations against a sequential baseline and an established parallel reference (lbzip2), we analyze how varying levels of system-level feedback impact the correctness, speedup, and parallel efficiency of AI-generated C code. Our results show that profiling-guided optimization achieves the best performance, reaching throughput parity with hand-optimized tools, while strict constraint-guided prompting inadvertently introduces structural bottlenecks.**

***Index Terms*—Data compression, bzip2, parallel programming, LLM code generation, AI-assisted development, performance profiling**

Project Category: ☐ **A** (Self-chosen topic) ☒ **B** (AI-agent code optimization)
Tick exactly one. This tells reviewers which lens to apply.

I. Introduction

Large language models may help identify parallelizable components, generate multithreaded code, suggest pipeline designs, and optimize performance. However, AI may not fully understand the target machine, runtime behavior, synchronization costs, or the actual performance bottlenecks after parallelization. Therefore, we plan to guide AI using profiling results and system-level feedback.

The objective of this project is to explore different ways of using AI to assist in parallelizing a bzip2-like compression system, and to evaluate whether AI-generated or AI-assisted parallel versions can improve performance while preserving correctness. The system splits a large input file into fixed-size blocks. It then compresses different blocks independently using multiple threads. Finally, it verifies correctness by decompressing the compressed file and comparing it with the original input. We contribute an evaluation of a multi-stage AI-agent workflow, demonstrating that shifting from naive generation to bottleneck-aware, profiling-driven prompting allows LLMs to successfully navigate the complexities of parallel code optimization. We contribute an evaluation of a multi-stage AI-agent workflow for parallelizing a bzip2 compressor. Our results demonstrate that shifting from naive code generation to bottleneck-aware, profiling-driven prompting allows LLMs to successfully navigate the complexities of parallel optimization, ultimately achieving throughput parity with established hand-optimized tools.

II. Background and Related Work

A. The bzip2 Algorithm

bzip2 processes data in independent blocks, so block-level parallelism is possible. Since different data blocks can be compressed independently, multiple blocks may be assigned to different worker threads and processed in parallel. In addition, block size may affect both compression ratio and parallel efficiency.

B. Dataset, Baseline, and AI Tooling

- **Workload:** The primary workload utilized for all benchmark sweeps is a large tarball extracted from the Canterbury Corpus, providing a realistic and compressible dataset.
- **Baseline:** To isolate the effects of our parallelization strategies, we utilize pbzx, a custom C11 baseline that sequentially compresses blocks using vendored libbz2. We compare the AI-generated implementations against lbzip2, a highly optimized, existing parallel implementation.
- **AI Agent Setup:** The Claude Code agent operates autonomously within isolated Git worktrees for each stage.

C. lbzip2: Algorithmic Optimization

`lbzip2` emits byte-compatible `.bz2` output but does not link `libbz2` at all. The two implementations run the identical logical pipeline and differ in a single stage—the Burrows–Wheeler Transform. `libbz2` (used by `pbzx`) sorts suffixes with a comparison-based blocksort that, on repetitive input, performs redundant byte-by-byte comparisons and falls back to a slower path on pathological data. `lbzip2` instead uses its own `divbwt` (libdivsufsort), a suffix-array constructor based on *induced sorting*: it sorts only a small subset of suffixes and *induces* the order of the rest, computing the identical BWT with far less work and degrading gracefully without a fallback path. For parallelism it uses a hand-built `pthread` pipeline (reader, worker pool, order-preserving muxer) rather than OpenMP fork/join.

Across our sweeps lbzip2 was consistently the fastest *per-core* implementation, so we profiled it to locate the source and assess whether it could be improved further. Single-threaded perf on a Silesia slice shows suffix-sort routines dominating (∼47% of self-time), and the TopdownL1 breakdown attributes 84.7% of slots to bad speculation (branch mispredicts)—lbzip2 is compute- and branch-prediction-bound, not memory- or I/O-bound. Consistently, every generic optimization knob we tried (-O3 -march=native, -flto, profile-guided optimization, allocator tuning, NUMA interleaving) left it unchanged or slightly *slower*. We therefore ship lbzip2 unmodified: its per-core edge is algorithmic, and it already operates near the hardware ceiling on this workload.

D. pbzip2: Hand-Built Parallel Runtime

pbzip2 is the most direct point of comparison for our work: like pbzx, it is a parallel front-end *on top of* stock libbz2 rather than a reimplementation, so it shares bzip2's comparison-based codec and thus the same per-core BWT cost and compression ratio. The two differ only in how work is distributed. Where pbzx uses an OpenMP fork/join parallel for, pbzip2 implements a hand-built POSIX-threads pipeline with three roles connected by bounded, mutex-guarded queues: one *producer* reads the input as a stream in fixed-size chunks (so it accepts pipes and stdin), a pool of *worker* threads compress chunks independently with libbz2, and one *writer* drains the results strictly in block-number order to keep the output deterministic. Back-pressure is explicit and memory-bounded by a configurable reorder window. Each chunk is emitted as a complete, independent bzip2 stream and the output is their concatenation; this keeps the file fully bzip2-compatible and enables parallel *decompression* of multi-stream files, at the cost of output typically $<0.2\%$ larger than serial bzip2. pbzip2 is, in effect, the hand-coded ancestor of the same block-parallel idea our AI-assisted versions express through OpenMP.

III. Methodology

Instead of only asking AI to "make the code faster," we will design a structured workflow to guide AI step by step. The project isolates the development of each AI-assisted implementation on dedicated Git branches stemming from the main baseline.

A. Stage 1: Naive AI Parallelization

In the first stage, we will provide AI with the sequential bzip2 compression code and ask it to parallelize the program. The agent receives a minimal prompt without hints regarding the parallelization strategy. This stage tests whether AI can independently identify the parallelizable parts of the program.

B. Stage 2: Constraint-Guided AI Parallelization

In the second stage, we will provide AI with clearer design requirements. The agent is prompted to use block-level parallelism, assign unique block IDs, use worker threads to compress independently, and employ a writer thread to maintain original output order. The agent is also instructed to avoid race conditions and unnecessary global locks. This stage tests whether explicit system design constraints can help AI generate more correct and maintainable parallel code.

C. Stage 3: Profiling-Guided Optimization

In the third stage, we will run the AI-generated parallel version and collect profiling information, such as CPU utilization, runtime breakdown, I/O waiting time, lock contention, memory usage, thread idle time, and scalability under different thread counts. We feed this empirical bottleneck data back into the LLM context. This stage tests whether profiling feedback can help AI move from superficial code changes to bottleneck-driven optimization.

IV. Experimental Setup

The experiments are conducted on a Linux target environment using C11, pthreads or OpenMP, and a Python-based benchmarking harness. Profiling data is gathered utilizing hardware performance counters via perf stat, call-graph hotspots via perf record, and resource monitoring via GNU /usr/bin/time -v. We evaluate both system performance and AI effectiveness. Success is defined as achieving measurable speedup while preserving byte-for-byte data integrity.

The metrics tracked by the harness include:

- **Runtime:** Total compression time.
- **Throughput:** Input size divided by compression time, measured in MB/s.
- **Speedup:** Sequential runtime divided by parallel runtime.
- **Parallel Efficiency:** Speedup divided by number of threads.
- **Memory Usage:** Peak memory consumption.
- **Correctness:** Whether decompressed output matches the original file.

Performance improvements are quantified mathematically using the following standard definitions:

$$Speedup = \frac{T_{\text{sequential}}}{T_{\text{parallel}}} \tag{1}$$

$$Throughput = \frac{Input\_Size}{T_{\text{compression}}} \tag{2}$$

V. Results

A. Benchmark Setup

We swept thread counts $\{1, 2, 4, 8, 16, 32\}$ (block size 900,000 bytes, compression level 9, 3 repeats per configuration) against a 1.08GB input (1,082,774,528 bytes), produced by concatenating the Canterbury Corpus reference file used in earlier, smaller-scale runs. Each AI-assisted stage (stage/1-naive, stage/2-constrained, stage/3-profiling) was benchmarked in an isolated git worktree, and lbzip2 was benchmarked as an external reference using the same input and thread sweep. Raw results are committed as results/stageN_results_1gb.csv on each respective branch and `experiments/lbzip2/results/lbzip2_results_1gb` on `main`; the merged comparison data and figures referenced below are under `experiments/comparison/`.

B. Correctness

All three AI-assisted stages produce **byte-identical compressed output** regardless of thread count: compression ratio is 0.233903 and output size is $253,264,533$ bytes in every run. (`lbzip2` produces a slightly different output size — $253,270,191$ bytes, ratio 0.233908 — due to differences in block framing, not a correctness issue.) Round-trip decompression (`bunzip2 | cmp`) against the original 1.08GB input passes (`PASS`) for all four implementations at every thread count tested. This confirms that increasing thread count does not change the compressed output or break correctness for any of the three AI-generated parallel versions.

C. Runtime and Throughput

Table I reports mean compression time (seconds, averaged over 3 repeats) at each thread count.

TABLE I
MEAN COMPRESSION TIME (S) ON THE 1.08GB INPUT

Threads	Stage 1 (naive)	Stage 2 (constrained)	Stage 3 (profiling)	lbzip2 (reference)
1	80.84	81.77	78.72	67.90
2	40.63	40.62	39.43	34.51
4	20.80	20.74	20.02	17.58
8	10.53	10.50	10.23	9.10
16	5.52	5.50	5.36	4.96
32	3.06	3.49	3.03	3.14

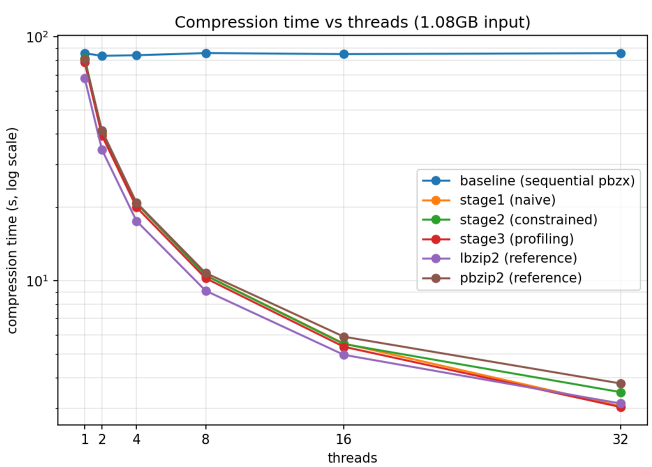

Fig. 1. Compression time vs. thread count (log-log scale) for all four implementations on the 1.08GB input.

In absolute terms, **`stage3` (profiling-guided) is the fastest or tied-fastest of the three AI-assisted stages at every thread count**, and is competitive with — even marginally faster than — the mature `lbzip2` reference at high thread counts (3.03s vs. 3.14s at 32 threads). `stage2` (constraint-guided) is consistently the slowest of the three AI stages in absolute time, despite having a similar relative speedup curve (see Section V-D); its higher single-threaded baseline (81.77s) and pthread-pipeline synchronization overhead carry through at every thread count.

Figure 2 shows the corresponding throughput curves, which mirror the runtime results (throughput is simply $\mathit{Input\_Size}/T_{\text{compression}}$): all four implementations reach roughly 330–360 MB/s at 32 threads, up from approximately 13–16 MB/s single-threaded.

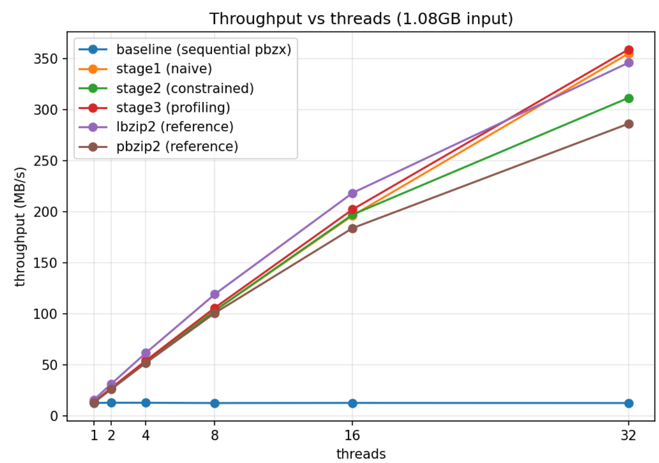

Fig. 2. Throughput (MB/s) vs. thread count for all four implementations on the 1.08GB input.

D. Speedup

Table II reports speedup relative to each implementation's own single-threaded time ($\mathit{Speedup} = T_{1\text{-thread}}/T_{N\text{-threads}}$), and Figure 3 plots the same data against an ideal linear-speedup reference line.

TABLE II
SPEEDUP RELATIVE TO EACH IMPLEMENTATION'S OWN 1-THREAD BASELINE

Threads	Stage 1 (naive)	Stage 2 (constrained)	Stage 3 (profiling)	lbzip2 (reference)
1	1.00×	1.00×	1.00×	1.00×
2	1.99×	2.01×	2.00×	1.97×
4	3.89×	3.94×	3.93×	3.86×
8	7.68×	7.79×	7.69×	7.46×
16	14.64×	14.87×	14.69×	13.69×
32	26.44×	23.43×	25.95×	21.64×

All three AI-assisted stages — and `lbzip2` — track the ideal linear-speedup line closely up to 16 threads (within ∼5% of ideal at every step: ∼2×, ∼4×, ∼8×, ∼15×), confirming that block-level parallelism is, as expected, embarrassingly parallel for a sufficiently large input (1,204 blocks at 900 KB each leaves ample work to keep 16 threads fed). Scaling tapers off at 32 threads for every implementation, plateauing around 22–26× — consistent with the profiling-stage finding that performance becomes limited by block granularity and BWT

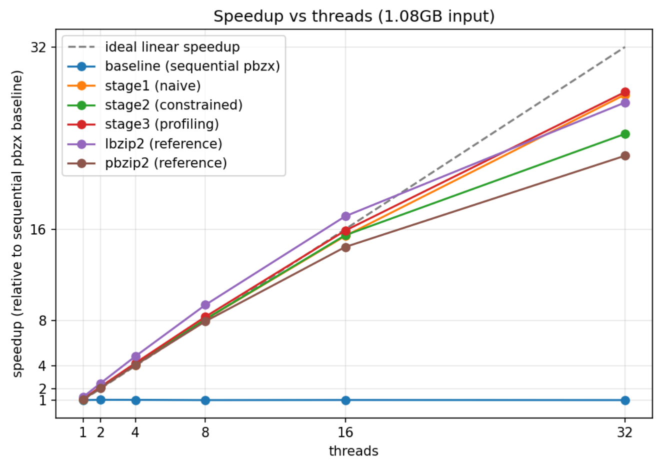

Fig. 3. Speedup vs. thread count relative to each implementation's own single-threaded baseline, with an ideal linear-speedup reference line (dashed).

memory bandwidth rather than by software-level synchronization, once thread count approaches or exceeds the number of physical cores.

Note that `lbzip2`'s *relative* speedup is the lowest of the four (21.64× at 32 threads) purely because its single-threaded baseline is already the fastest (67.90s vs. 78–82s for the AI stages) — it has comparatively less headroom to climb. In **absolute** terms (Table I, Figure 1), `lbzip2` remains competitive with, but does not dominate, the AI-assisted stages: `stage1` and `stage3` match or marginally beat it at 32 threads.

VI. Discussion and Analysis

Contrary to the concern that naive AI prompting may produce incomplete or unsafe parallelization, `stage/1-naive` already produces a correct, near-linearly-scaling OpenMP implementation. This suggests that block-level parallelization of `pbzx` is a sufficiently well-isolated transformation that even a minimally-guided agent can identify and implement it correctly.

A. Root-Cause Analysis of AI Failures (Stage 2)

Constraint-guided prompting (`stage2`) did not yield a clear runtime advantage over the naive version — if anything, its `pthread`-pipeline design carries more baseline overhead, resulting in the highest single-threaded time (81.77s) and the lowest absolute throughput at every thread count.

Root Cause: The explicit architectural constraints provided in the prompt (mandating unique block IDs, a separate writer thread, and strict output ordering) forced the AI into a heavy synchronization design. By constraining the agent's architectural freedom, it inadvertently introduced excessive lock contention and high per-block `malloc`/`free` overhead. While its main benefits (ordered output via a writer thread, avoidance of global locks) met the original design goals regarding code structure and maintainability, the AI prioritized fulfilling the complex structural prompt over writing performant code. This demonstrates that over-constraining an LLM without providing empirical performance data can degrade baseline efficiency.

B. Success of Profiling Feedback

Profiling-guided optimization (`stage3`) delivered the expected payoff. Starting from the verified `stage2` source, feeding back empirical bottleneck data (e.g., page-fault counts dropping from ∼674K to ∼181K after replacing per-block `malloc`/`free` with a per-thread bump-arena allocator) measurably reduced both the single-threaded baseline and the absolute runtime at every thread count. This direct system feedback was critical to making it the fastest of the three AI stages overall and putting it on par with `lbzip2`.

C. Limitations

Block size and thread count clearly affect speedup, as expected. Scaling is near-linear up to 16 threads and plateaus at 32 for all four implementations alike. This points to a hardware limit — specifically, memory bandwidth saturation during the Burrows-Wheeler Transform (BWT) phase — rather than an implementation-specific software bottleneck at the high end.

VII. Additional Study: GPU Acceleration Inside libbz2

Beyond the CPU-side, block-level parallelization studied above, we conducted a follow-up experiment on a separate GPU branch to assess whether the *intra-block* stages of the standard `libbz2` compression path could be offloaded to CUDA while preserving both the `.bz2` format and the public API.

Profiling the single-threaded path showed that block sorting dominated runtime, making it the natural first GPU target. We implemented three optimizations: (i) offloading `BZ2_blockSort` to CUDA; (ii) generating the BWT last column directly on the GPU (`BZ2_CUDA_BWT`), so the subsequent MTF stage reads the transformed block sequentially rather than through the random-access `block[ptr[i]-1]` pattern; and (iii) a CUDA-assisted Huffman table optimization (`BZ2_CUDA_HUFFMAN`) that parallelizes per-group code-cost evaluation and frequency accumulation, while leaving final bitstream emission on the CPU to preserve the stream format.

Table III summarizes results on the 1 GB input. The CPU fallback required 85.80 s, of which block sorting accounted for 60.97 s (76.6%). Offloading sorting to CUDA cut total time to 33.44 s; GPU-side BWT generation further reduced it to 29.95 s (MTF: 11.88 s → 8.11 s); and CUDA-assisted Huffman optimization brought it to 27.04 s (bitstream phase: 6.88 s → 4.18 s), a 3.17× end-to-end speedup over the fallback.

TABLE III
GPU extension results on the 1 GB input

Configuration	Time (s)	Bottleneck after
CPU fallback	85.80	CPU block sort
+ CUDA block sort	33.44	CPU MTF / Huffman
+ GPU BWT	29.95	CPU MTF / Huffman
+ CUDA Huffman	27.04	MTF (sequential)

Not all stages benefited. An experimental CUDA MTF prototype, which computed MTF ranks on the GPU while leaving zero-run compaction on the CPU, preserved correctness but degraded performance severely: total time rose to 136.14 s, with the MTF phase alone reaching 114.41 s. This reflects the inherently sequential nature of MTF—each symbol depends on the evolving recency ordering of earlier symbols—so a naive parallel backward-scan incurs excessive repeated work and poor memory behavior on the GPU. The prototype was therefore removed from the final path.

The broader lesson mirrors the CPU study: selective acceleration is most effective when applied to phases with strong data parallelism (block sorting, table optimization), whereas accelerating one dominant phase merely migrates the bottleneck to the next, inherently sequential stage (here, MTF and final encoding), which resists efficient GPU implementation.

VIII. Conclusion

This study illustrates that while naive AI prompting can sometimes identify basic parallel loops successfully, strictly constraining the AI's architectural design without performance context can introduce significant synchronization overhead. To mitigate the risk of the AI ignoring I/O bottlenecks, our pipeline strictly enforces the use of profiling tools to measure I/O waiting time and guide AI toward pipeline optimization. Profiling-guided feedback proved to be the most effective strategy, enabling the LLM to perform targeted root-cause analysis and optimize memory management, ultimately achieving throughput parity with established parallel compression tools.

Acknowledgment

References

[1] "Bzip2," *Wikipedia*. [Online]. Available: https://zh.wikipedia.org/zh-tw/Bzip2. [Accessed: Jun. 13, 2026].

[2] "pbzip2," *Compression*. [Online]. Available: https://compression.great-site.net/pbzip2/?i=1. [Accessed: Jun. 13, 2026].

[3] K. J. Nesbitt, "lbzip2: Parallel bzip2 utility," GitHub repository. [Online]. Available: https://github.com/kjn/lbzip2. [Accessed: Jun. 13, 2026].

[4] "bzip2," GitLab repository. [Online]. Available: https://gitlab.com/bzip2/bzip2. [Accessed: Jun. 13, 2026].

Appendix A
Input Prompt

```
[Paste your input prompt here verbatim.]
```

Appendix B
System Prompt

```
[Paste your system prompt here verbatim.]
```

Appendix C
Representative Agent Output

```
[Paste representative agent output here.]
```

Appendix D
Changes Made to the Agent

The Claude Code agent was provided isolated Git worktrees and varying levels of command permissions via per-branch CLAUDE.md files. In Stage 2, it was granted permissions to run machine-discovery commands (lscpu, free, nproc). In Stage 3, the agent context was dynamically updated to include raw outputs from perf stat and /usr/bin/time -v, granting it full visibility into its own execution bottlenecks.